Dynamics and Control - RMan project 2

Mathias Dam - ist1117164 Timothée Lawani - ist1119605 Martin Chodos Figueiredo - ist1119554

This notebook presents answers to the questions in the second part of the RMan project. The project was implemented using Python code. This report is an export of the jupyter notebook file that the group worked in to provide answers to all questions. The group decided to submit this instead of a standard pdf report, as it contains detailed answers to all questions and all the code used in a single file, making evaluating easier for the professor.

In addition to this pdf, a zipped folder with all the code done for the project is included in the submission, including the jupyter notebook that this pdf is made from. All the libraries used are stored in a virtual enviorment inside the folder, so the professor will be able to run the jupyter notebook, verify the groups code and get interactive 3d-animations of the robot arm movement without installing any libraries. Two video files are also submitted, containing screen recordings of the animations of the robot arm following the path from question 5 using the controllers implemented in question 3 and 4. This lets the professor see the animations without running the code.

Summary

This notebook implements a full dynamic simulation of a **KUKA LBR Med 7 R800** 7-DOF robot arm, covering:

1. **Link inertia estimation** — each link approximated as a solid cylinder, calibrated to the 24 kg catalogue mass.
2. **Dynamic model** — Newton-Euler forward dynamics: inertia matrix B(q), Coriolis vector C(q,q̇)q̇, and gravity vector g(q), combined as $\ddot{q} = B^{-1}(\tau - C\dot{q} - g)$. An optimised vectorised implementation reduces computation from ~18 ms to ~6 ms per call.
3. **Decentralized PID control** — per-joint PID with critically damped gains tuned at the worst-case inertia configuration.
4. **Computed Torque Control (CTC)** — full inverse-dynamics feedforward with proportional-derivative error feedback, giving near-perfect tracking in the ideal model.
5. **Task-space trajectory tracking** — a Cartesian end-effector path (XY square → XZ square → XZ circle) is converted to joint space via CLIK (Closed-Loop Inverse Kinematics), then both controllers are compared on position tracking error.

Library Imports and Variable Definitions

Question 1 - Approximate Link Inertias

Method

Each of the 7 links is approximated as a **single solid cylinder** sized to the arm's outer envelope. Lengths come from the published iiwa kinematic offsets (0.34 / 0.40 / 0.40 / 0.126 m for the limb segments; the shoulder, elbow and wrist joint-pairs are co-located, so their housings are modelled as short cylinders). Radii are read off the photograph, tapering from ~65 mm at the base to ~35 mm at the flange.

A real cobot link is a hollow aluminium shell containing a motor, gearbox and torque sensor. Instead of modelling those internals, the solid cylinder is given a single **effective density** $\rho = 1565$ kg/m³ (~58% of solid aluminium). That value is calibrated so the seven cylinder masses sum to the catalogue robot mass of ~24 kg. Using the larger cylinders for the proximal links automatically puts more mass near the base, as in the real arm.

Link frame *i* sits at joint *i* with its **z-axis along the joint axis**. Since each cylinder is axisymmetric about that axis, every COM lies on z and each inertia tensor is **diagonal** in the link frame (products of inertia ≈ 0).

Formulas

For a cylinder of mass $m = \rho \cdot \pi r^2 L$, length L along its axis, radius r:

- axial inertia: $\mathbf{I_{zz} = \tfrac{1}{2}\, m\, r^2}$
- transverse inertia: $\mathbf{I_{xx} = I_{yy} = \tfrac{1}{12}\, m\, (3r^2 + L^2)}$

both taken about the COM, located at the cylinder's mid-length.

Per-link results

Link	Calculation (L, r → m; Izz; Ixx=Iyy)
1 base/shoulder	L=0.34, r=0.065 → m=7.06 kg; Izz=0.0149; Ixx=Iyy=0.0755; com_z=0.150
2 shoulder pitch	L=0.18, r=0.060 → m=3.19 kg; Izz=0.00573; Ixx=Iyy=0.0115; com_z=0
3 upper arm	L=0.40, r=0.055 → m=5.95 kg; Izz=0.00900; Ixx=Iyy=0.0838; com_z=0.180
4 elbow pitch	L=0.16, r=0.050 → m=1.97 kg; Izz=0.00246; Ixx=Iyy=0.00542; com_z=0
5 forearm	L=0.40, r=0.045 → m=3.98 kg; Izz=0.00403; Ixx=Iyy=0.0551; com_z=0.170
6 wrist pitch	L=0.14, r=0.040 → m=1.10 kg; Izz=0.000881; Ixx=Iyy=0.00224; com_z=0
7 flange/tool	L=0.126, r=0.035 → m=0.76 kg; Izz=0.000465; Ixx=Iyy=0.00124; com_z=0.050

Units: kg, m, kg·m². Total mass = 24.0 kg.

Python representation

Question 2 - Making the model

We need to make a model. This basically means that we need to make some sort of code where we can test out different functions for calculating the joint torque vector, then we need to be able to calculate the resulting motion of the whole robot arm and plot it and evaluate it.

Framework

Fundamentally, this model will be based on a function that takes in the current state, current timestep, and some sort of function for that returns the torque vector, and then returns the velocity and acelleration, like this template function:

```
def dynamics(t, q, q_dot, tau_func):
    tau = tau_func(t, q, q_dot)
    q_ddot = forward_dynamics(q, q_dot, tau)
    return q_dot, q_ddot
```

This function can then be solved with an ODE solver for example from scipy to get the full behaviour of the robot arm. Saving an array with the states at the different timesteps will allow us to easily plot the movement of the robot arm.

Implementation of the forward_dynamics function

Now that we have a plan for the framework, let's start diving into the requirements for implementing the function above. The tau function will be based on what control algorithm is used, so this will be dealt with in question 3 and 4, for now we can just ignore it. The forward_dynamics function we need to implement now. This function will be based on the equation:

$$\tau = B(q)\ddot{q} + C(q,\dot{q})\dot{q} + g(q)$$

This equation can easily be rewritten so that $\ddot{q}$ stands alone, giving us exactly what we need for the forward_dynamics function. However, we need to find the expressions for $M(q)$, $C(q,\dot{q})$ and $g(q)$. LEt's find these terms one by one.

Inertia matrix

The mass and inertia matrix $B(q)$ can be found either by using a Lagrangian approach, or by using the newton-euler method. We will now find it using the newton-euler method because it is slightly more efficient, and because it is harder to implement so we will learn more by implementing it.

Row n in B(q) represents all the torques required to accelerate joint n. When we use the Newton-Euler approach, we find each of these columns by doing a forward and backwards pass. We start by assuming that all velocities and gravity are zero. Then, in the forwards pass, we calculate how each link will move

when we have an acceleration on joint n. To calculate this, we use formulas based on Newtons equations. For a revolute joint, the forward Newton--Euler pass can be written as:

$$\boldsymbol{\omega}_i = {}^iR_{i-1}\boldsymbol{\omega}_{i-1} + \mathbf{z}_i\dot{q}_i$$

$$\dot{\boldsymbol{\omega}}_i = {}^iR_{i-1}\dot{\boldsymbol{\omega}}_{i-1} + \mathbf{z}_i\ddot{q}_i + \left({}^iR_{i-1}\boldsymbol{\omega}_{i-1}\right) \times \left(\mathbf{z}_i\dot{q}_i\right)$$

$$\mathbf{a}_i = {}^iR_{i-1}\left(\mathbf{a}_{i-1} + \dot{\boldsymbol{\omega}}_{i-1} \times \mathbf{p}_i + \boldsymbol{\omega}_{i-1} \times \left(\boldsymbol{\omega}_{i-1} \times \mathbf{p}_i\right)\right)$$

$$\mathbf{a}_{c_i} = \mathbf{a}_i + \dot{\boldsymbol{\omega}}_i \times \mathbf{r}_{c_i} + \boldsymbol{\omega}_i \times \left(\boldsymbol{\omega}_i \times \mathbf{r}_{c_i}\right)$$

Then, in the following backwards pass we use another set of formulas based on newtons laws to calculate the torque felt in each joint based on the accelerations of the link that comes after the joint and the force that is transmitted from the next joint. {Claude, help me out and insert formulas here}

Below is an implemented frunction B(q) that is based on this method.

```
B symmetric: True
Eigenvalues (all positive?): [5.000e-04 1.570e-02 3.970e-02 1.109e-01 1.639e-01 1.157e+00 6.2
05e+00]
```

Coriolis vector

Next, we need to find the coriolis term $C(q, \dot{q})\dot{q}$. To find this, we can actually use the Newton-Euler method again. We do a very similar implementation as the one above, but instead of setting the gravity and velocity to zero, we set the gravity and *acceleration* to zero.

We use the same formulas that we used to find B(q). An implementation of a C_vector(q, q_dot) function using this approach can be found below.

```
C_vector(q, 0): [0. 0. 0. 0. 0. 0. 0.]
```

Gravity vector

The last component needed to finish our masterplan from above is the gravity vector. This vector simply says what torque acts on each joint due to gravity pulling the robot arm down.

For the gravity vector we actually still use the Newton-Euler approach, just that now we set both velocity and acceleration to zero, and only include the gravity term in the input. This is implemented below.

```
g(q=0): [0. 0. 0. 0. 0. 0. 0.]
```

Full forward dynamics function

The three terms that we have now calculated can all be combined to get the full forward dynamics function by using the equation:

$$\ddot{q} = B(q)^{-1}(\Tau - C(q, \dot{q})\dot{q} - g(q))$$

Below is an implementation of the full forward dynamics function.

Full dynamic simulation function

Now that we have implemented the forward dynamics function, we are ready to build the full dynamic model. First, we build the single timestep dynamic evaluation function:

The output values from the tau function are clipped. This is to simulate that the actuators in the robot-arm have a maximum torque. The values are estimates on how much torque each joint in the robotic arm is able to produce.

Then, the full solution is obtained by solving this function with an ODE solver:

In the code in the following questions, the rtol and atol input parameters to the simulation function are set to 1e-3 and 1e-5 respectively. This is to speed up the differential equation solver, to reduce the time it takes to run the model. If more precision is needed, these parameters can be set to lower values.

Plotting

In order to get any useful information from the simulation, we need to be able to plot the results. Below are two functions for doing this, both take the output from the simulate function as input.

The first is more of a visual plot, it creates a moving 3d visual of the movement of the robot. Useful for seeing the movement and seeing if it looks correct. It is implemented using the library plotly.

The second one is a simple 2d plot showing the euclidean distance between the coordinates of the end effector and the desired end effector coordinates as a function of time, in addition to printing values like average and max error. This one is more useful for comparing the performance of different control algorithms. It is implemented using matplotlib.

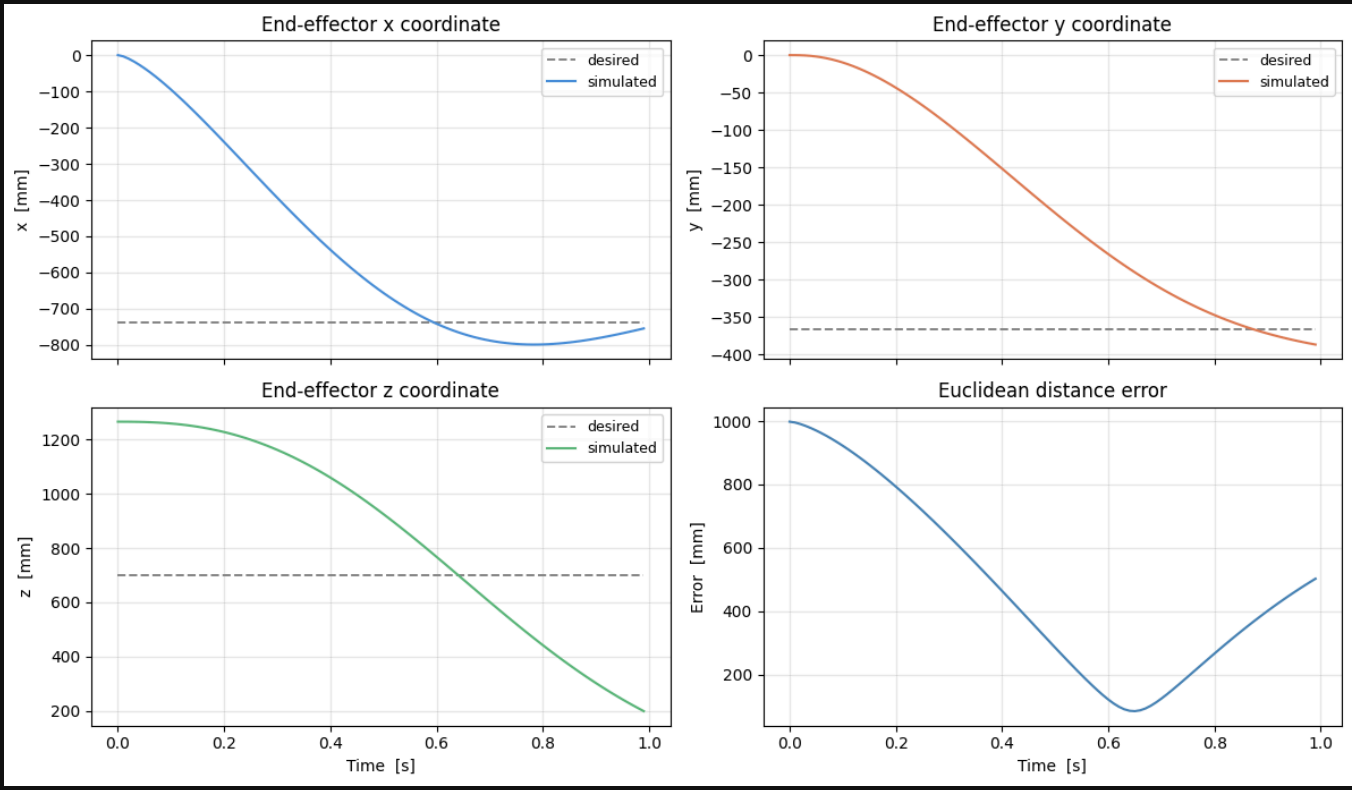
Validation

Before moving on to the next question, we should validate that the model we have implemented works. To do this, we will implement a super simple PD-based tau function. This will not be properly tuned, and is therefore unlikely to give good results, but it is just a super simple implementation to see that the model can run and plot without issues, and that the robot arm behaves in a somewhat realistic way.

With this simple PD controller implemented, we can run the simulation with 0 on all join angles as the initial position and see how it behaves, plotting the results.

```
Simulating: 100%|██████████| 1.0/1 [02:12<00:00, 132.68s/s]
```

```
End-effector position error:
  Final : 502.38 mm
  Mean  : 477.91 mm
  Max   : 998.32 mm
```



Question 3

First, the worst-case inertia configuration is selected. This is when the arm is fully extended horizontally:

$$q^* = [0,\ \pi/2,\ 0,\ 0,\ 0,\ 0,\ 0]$$

Decentralized PID Joint Controller

Principle

A decentralized controller treats each joint as an independent SISO system. Each joint i gets its own PID loop, ignoring coupling with other joints. This is computationally cheap and straightforward to implement, but does not compensate for gravity, Coriolis, or inertia coupling between joints.

Control Law

$$\tau_i = K_p\, e_i + K_i \int e_i\, d\xi - K_d\, \dot{q}_i$$

where $e_i = q_{d,i} - q_i$ is the position error for joint i.

Gain Selection — Critical Damping

To select the gains, each joint i is modelled as a single-DOF mass–damper system driven by its diagonal inertia $B_{ii}(q^*)$ evaluated at the worst-case configuration. Critical damping ($\zeta_i = 1$) is achieved with:

$$K_d = 2\sqrt{K_p\, B_{ii}(q^*)}$$

This makes the effective natural frequency $\omega_{n,i} = \sqrt{K_p / B_{ii}(q^*)}$ depend on the joint inertia, meaning heavier joints converge more slowly for the same K_p.

Anti-Windup

The integral state is clipped to $[-e_{\max},\, +e_{\max}]$ with $e_{\max} = 5$ rad·s, preventing integrator wind-up during large transients.

Torque Saturation

Commanded torques are hard-clamped to ± 500 Nm, within the KUKA LBR Med 7 actuator limits.

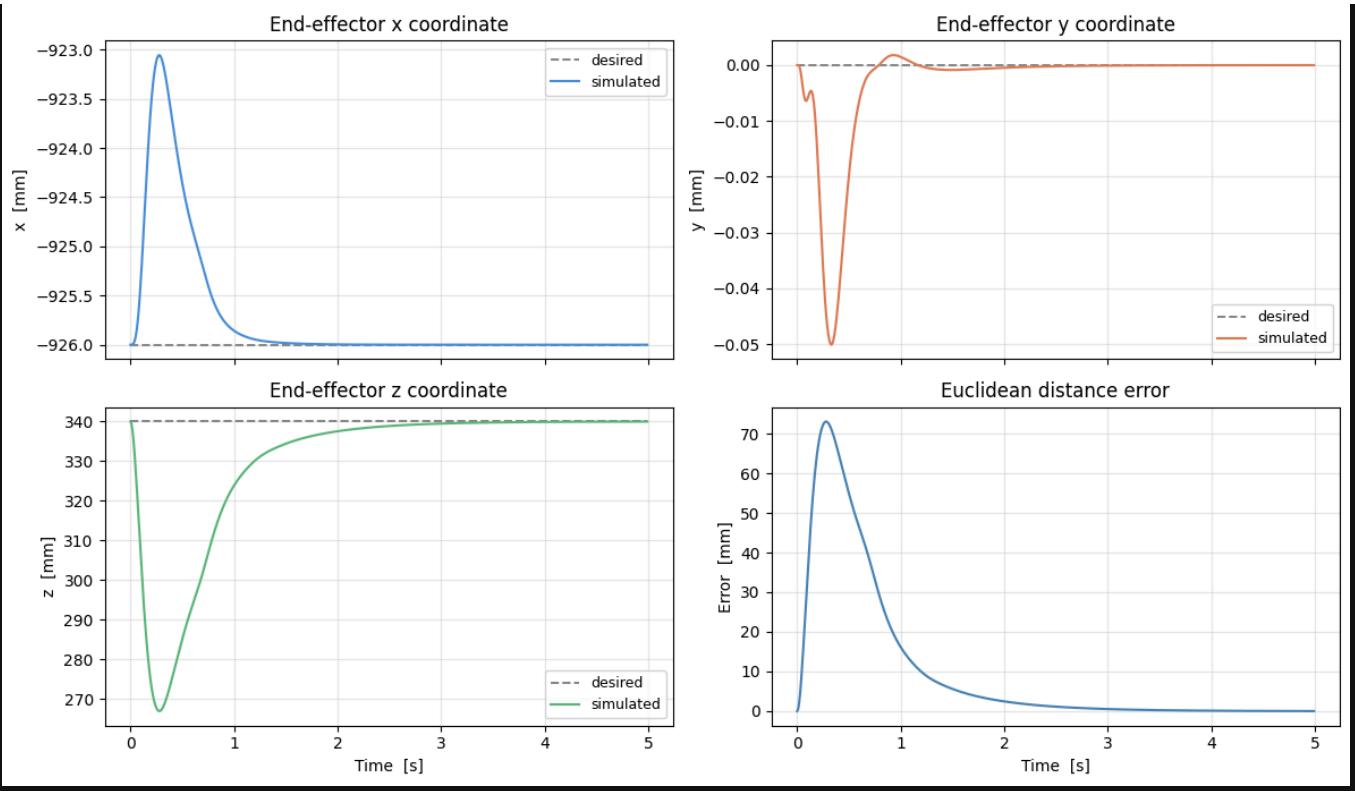
Limitations

- **Gravity**: With no feedforward gravity term, the integrator must wind up to compensate for gravity, introducing a transient offset before steady-state is reached.
- **Coupling**: Cross-joint inertia (B_{ij}, $i \neq j$) and Coriolis terms are unmodelled; at high speeds or in complex configurations these cause tracking errors the decentralised PID cannot anticipate.
- **Configuration-dependence**: Gains are tuned at q^*; far from this configuration $B_{ii}(q)$ may differ significantly, altering the effective damping ratio.

Implementation

The code below implements the PID controller described above, and validates it by running the model with the controller trying to hold the arm in a stable stretched-out position.

```
Simulating: 100%|██████████| 5.0/5.0 [01:37<00:00, 19.49s/s]
End-effector position error:
  Final : 0.02 mm
  Mean  : 10.36 mm
  Max   : 73.04 mm
```



From the plots it can be observed that gravity causes an initial error as the arm is pulled down, but the I-term soon builds up and recovers the arm to the desired position.

Question 4

For question 4, we need to implement a centralized inverse dynamics controller for the robot arm. We will implement joint-space computed torque control (CTC).

This controller is based on the exact same equation that we used for defining the model in question 2:

$$\tau = B(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q)$$

To control the acceleration of the joints directly, taking gravity, inertia and coriolis effects into account, we simply set the torque we give to the motors to be decided by the equation above, by setting:

$$\tau = B(q)\ddot{q}_r + C(q, \dot{q})\dot{q} + g(q)$$

Where $\ddot{q}_r$ is the desired reference acceleration. Inserting this torque expression into the equation above describing the dynamics of the robot arm we get:

$$B(q)\ddot{q}_r + C(q, \dot{q})\dot{q} + g(q) = B(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q)$$

Which simplifies to:

$$\ddot{q}_r = \ddot{q}$$

Assuming that there is no noise or external forces except what is modelled in the dynamics equation, the joints acceleration perfectly matches our desired acceleration!

But still, one problem remains. We could just match the desired acceleration to the acceleration of our desired path, but noise and errors would accumulate and the position of the robot arm would drift away from the desired position. We would have a useless controller. Therefore, we need some clever way to find a desired acceleration that takes the position and velocity into account to lead us to our actual desired state - a feedback loop.

We will implement this feedback loop using the equation:

$$\ddot{q}_r = \ddot{q}_d + K_p\, e + K_d\, \dot{e}$$

Where the K's are gain matrices we need to tune, e is the position error and $\dot{e}$ is the velocity error. The errors are defined as:

$$e = q_d - q$$

$$\dot{e} = \dot{q}_d - \dot{q}$$

Here is the code implementation of the $\ddot{q}_r$ function:

If we insert the feedback loop equation to the simplified dynamics equation, we get:

$$\ddot{q} = \ddot{q}_d + K_p\, e + K_d\, \dot{e}$$

Which can be rewritten to:

$$K_p\, e + K_d\, \dot{e} + \ddot{e} = 0$$

We see that the error dynamics of the robot arm now act as a standard mass-damper system, where we can choose the coeficcients ourself to get the desired behaviour.

Normally, we chose $\zeta = 1$ by setting $K_p = \omega_n^2$ and $K_d = 2\omega_n$. However, we still have the omega parameter to play with. The higher we set this parameter, the faster the robot arm will converge to the desired position. In our theoretical model, we would get the best possible result by increasing ω_n as much as possible. However, if we increase it too much for a real application the actuators will not be able to provide the required torque, and the robot arm becomes more sensitive to inaccuracies in the dynamical model. We chose to set $\omega_n = 12$. This gives the following values for the tuning constants:

$$K_p = \omega_n^2 \mathbf{I} = 144\mathbf{I}, \quad K_d = 2\omega_n \mathbf{I} = 24\mathbf{I}$$

Below is the code implementation of this, the constants are implemented as diagonal matrixes with the tuning constant along the diagonal:

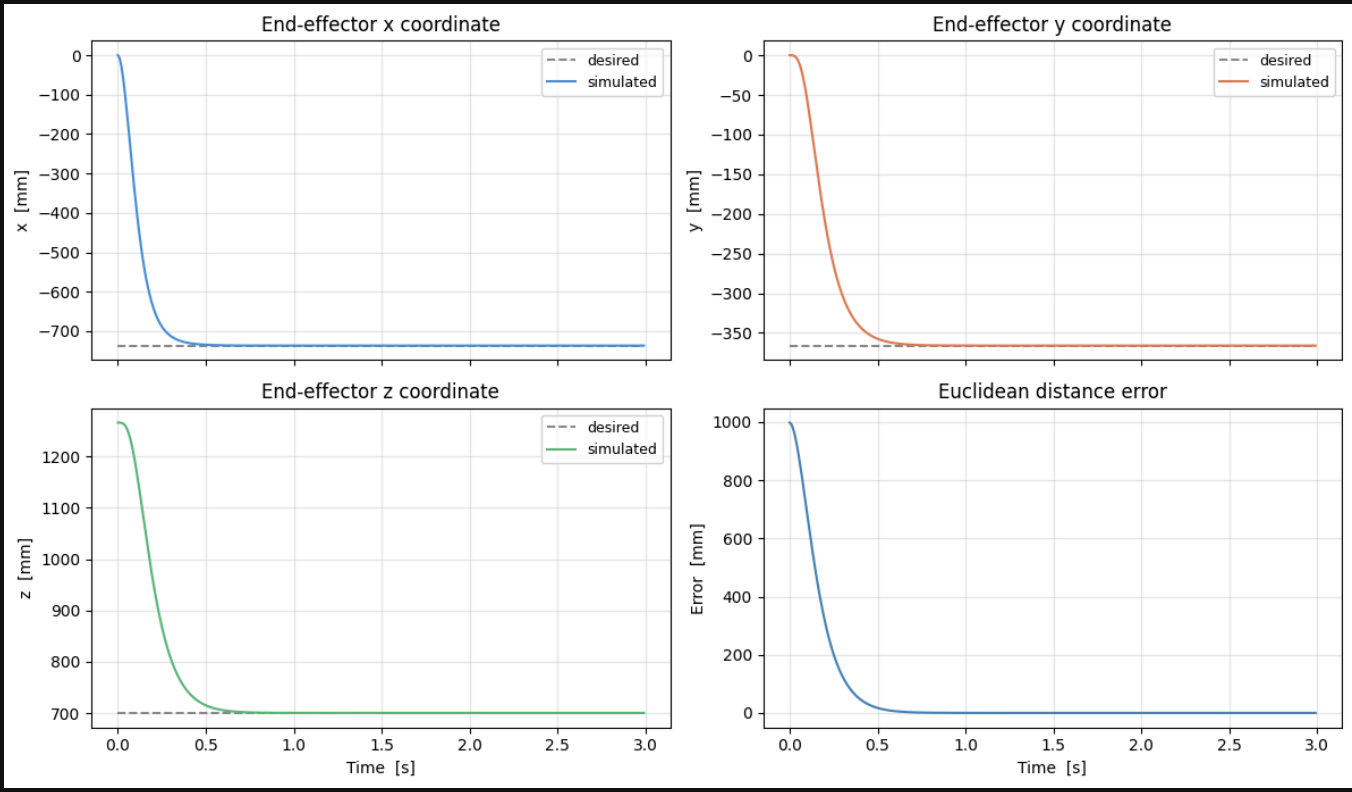
Full control function implementation

With all the prequisites in place, we can implement the full function for the CTC algorithm:

Validation

To validate that the CTC algorithm works as expected, we test it on a simple path:

```
Simulating: 100%|██████████| 3.0/3 [00:10<00:00,  3.37s/s]
End-effector position error:
  Final : 0.00 mm
  Mean  : 58.17 mm
  Max   : 998.32 mm
```



Question 5

Comparison: Decentralized PID vs Computed Torque Control

In this question, the PID and the CTC algorithms are compared on a complicated end-effector path.

End effector path

The first step is to define an end effector path in the task space that will be used to compare the two controllers. This path is constructed by specifying points that the end-effector should travel through, then using linear interpolation to interpolate between the points. Linear interpolation is used instead of a spline, as it preserves the straightness of the segments for rectangular movements. This also creates acceleration spikes at corners, which will be interesting to see how the different controllers handle.

The path includes the robot's end effector first moving in a square pattern in two planes, then moving linearly up and down, then in a circle in the XZ-plane. The full motion takes about 8.2 seconds.

End effector orientation

In addition to the position path, the orientation of the end effector is specified for each segment:

- **XY square** (t = 0–1.6 s): The end effector points perpendicular to the XY plane — its z-axis points straight down along the world Z axis.
- **XZ square** (t = 2.0–3.2 s): The end effector points perpendicular to the XZ plane — its z-axis points in the −Y direction.
- **Up-down** (t = 3.6–5.4 s): The end effector points straight down, same orientation as during the XY square.
- **Circle** (t = 5.8–8.2 s): The end effector z-axis points toward the centre of the circle at all times, rotating continuously as the arm traverses the circle.

Between each segment the end effector holds its position at the path centre while the orientation smoothly transitions over 0.4 seconds using SLERP (Spherical Linear Interpolation). The angular feedforward velocity is passed to the CLIK reference velocity for smooth tracking.

```
EE centre: x=0.260  y=-0.260  z=0.218 m
C:\Users\Mathias\AppData\Local\Temp\ipykernel_6536\188222695.py:50: RuntimeWarning: divide by zero encountered in divide
  _v_seg = np.diff(p_wpts, axis=0) / np.diff(t_wpts)[:, None]  # (N-1, 3)
C:\Users\Mathias\AppData\Local\Temp\ipykernel_6536\188222695.py:50: RuntimeWarning: invalid value encountered in divide
  _v_seg = np.diff(p_wpts, axis=0) / np.diff(t_wpts)[:, None]  # (N-1, 3)
```

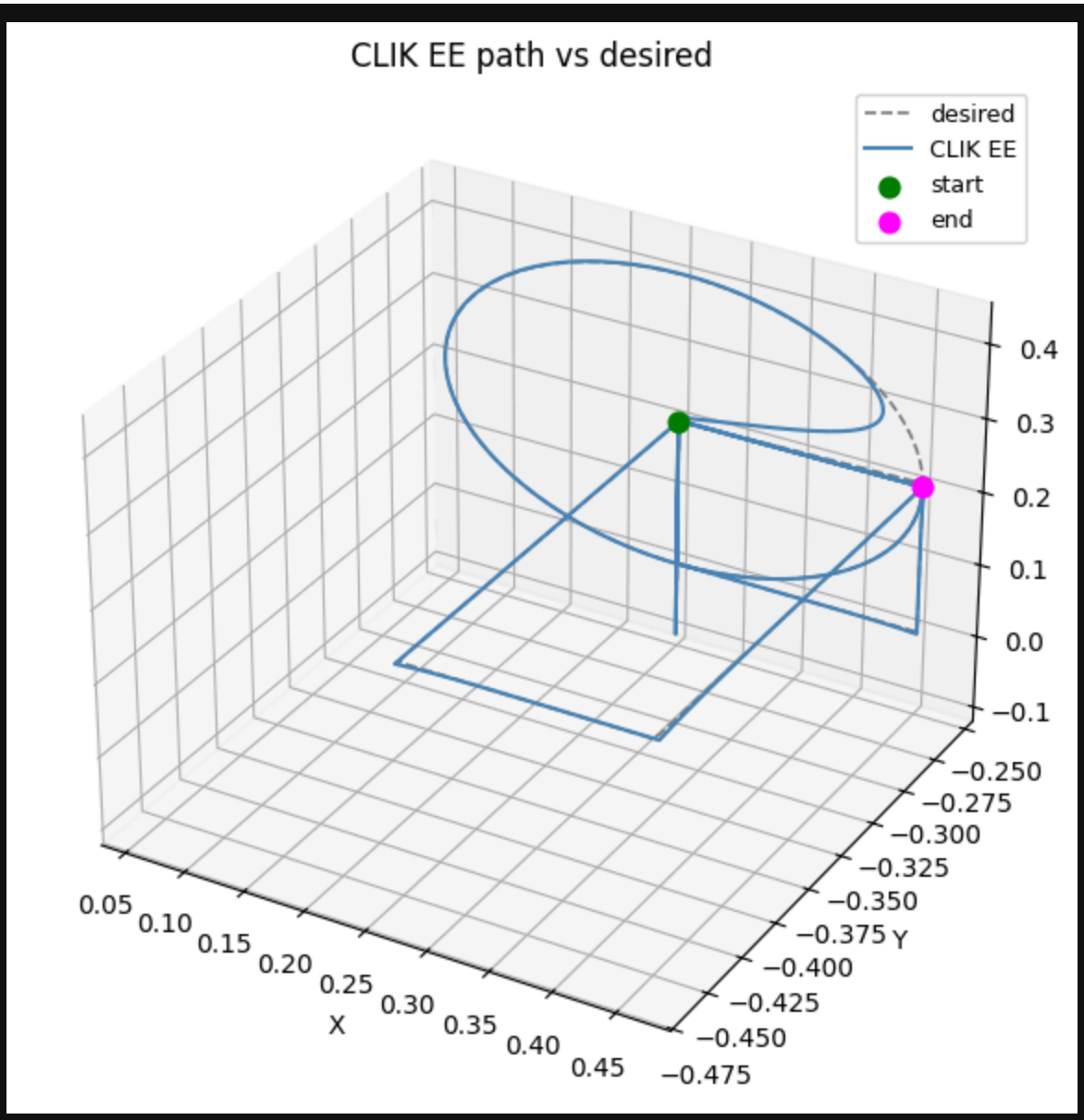
From task-space end effector path to joint-space trajectory

In order to simulate the controllers, we need to convert this task-space end effector trajectory into a joint-space trajectory. This is done using the CLIK algorithm that was implemented in the kinematics project earlier in the semester. The acceleration of the joint is also calculated by differentiating the velocity of the joints.

```
Generating CLIK trajectory...
CLIK: 100%|██████████| 821/821 [00:01<00:00, 415.24it/s]
```

Verifying Path

To verify that the CLIK path is valid, a 3d-animation is made of the robot arm perfectly following the path. If we see any difference between the desired path and the CLIK EE, that there is a singularity or out of bounds issue.




```
CLIK position error — mean: 2.14 mm   max: 200.02 mm
```

Running the simulation

The simulate function implemented earlier is called to simulate how the two controllers act, giving the joint-soace paths as input.

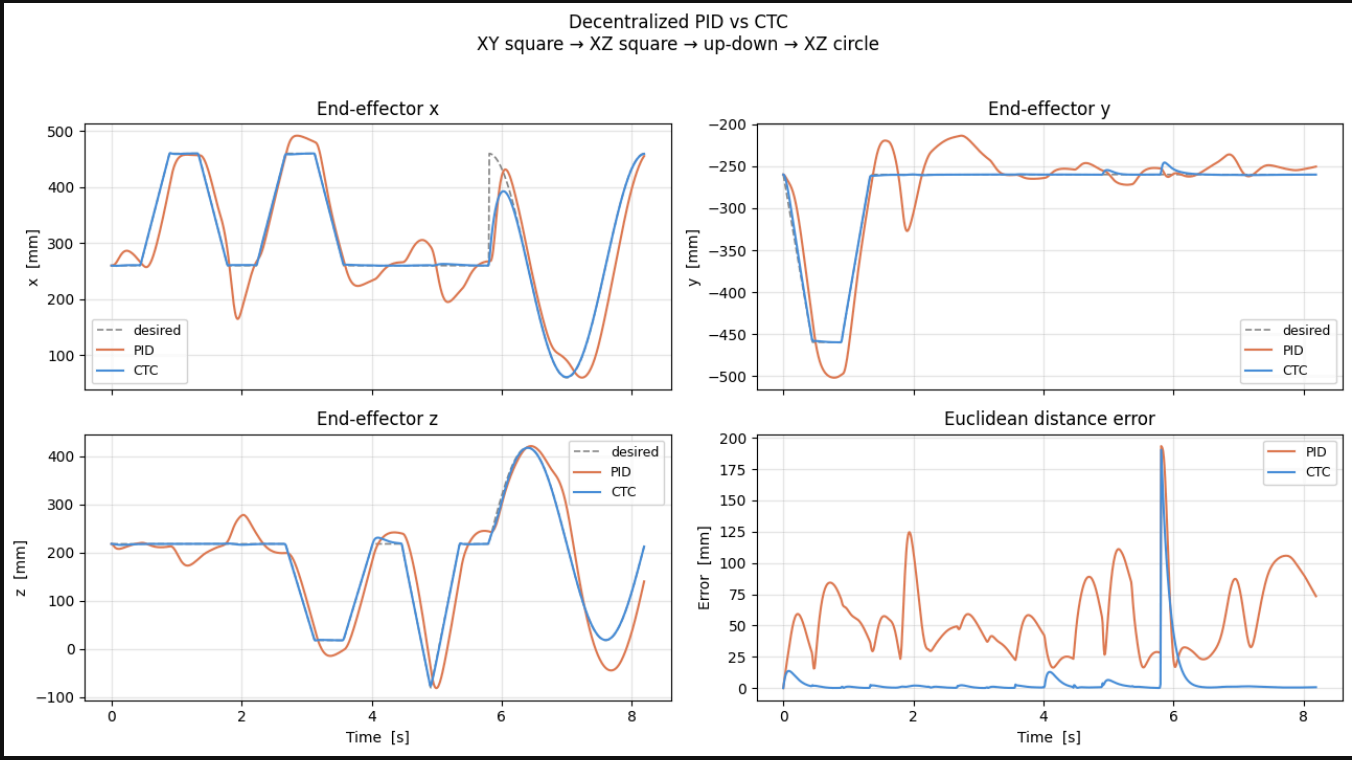
```
Simulating: 100%|██████████| 8.2/8.2 [05:40<00:00, 41.48s/s]
Simulating: 100%|██████████| 8.2/8.2 [00:36<00:00,  4.42s/s]
```

Plots

The results from both controllers are plotted in a 2d-plot showing their respective x, y and z positions along with the desired positions, and an error plot showing the euclidian distance between the desired end-effector position and the position the controller generates.

The rotation of the end-effector is not evaluated in the plots, as it is easier to get an intuitive understanding for the end effector coordinate plots, and these plots are sufficient to compare the controllers.

```
PID — final: 73.41 mm   mean: 56.18 mm
CTC — final: 0.70 mm   mean: 4.63 mm
```




```
=== PID Controller — 3D ===
=== Computed Torque Control — 3D ===
```

Analyzing results

From the 2d-plots we can clearly see that the CTC controller performs better than the PID controller.

The CTC controller is almost directly on the desired coordinates most of the time, with only small deviations caused by insufficient torque in the actuators to provide the required acelleration at sharp corners in the path. This is a hardware limitation, and would have to be solved by designing a path with continous velocity, for example using splines.

The PID controller lags behind the desired coordinate when it changes, and overshoots the desired position due to buildup of the I-term.

We can also observe that it deviates from the desired y-coordinate in the second half of the trajectory, even though the desired trajectory just has a constant y-position for this timespan. This is because it is a decentralized controller. All joints act independently, and they converge with different speeds, leading to deviations from the desired coordinate even though the desired coordinate does not move.

The spike in the error of both controllers at just before 6 seconds is caused by an irregularity in the path, demanding an instant change of the x-coordinate of the end effector. We can see that both controllers

are able to recover quickly from the error introduced by this irregularity.

The values for the mean and final error of the PID and CLIK controller are given below:

PID — final: 73.41 mm mean: 56.18 mm

CTC — final: 0.70 mm mean: 4.63 mm

It is worth noting that the simulation only simulates ideal contitions with no noise or friction. Adding noise and friction to the model simulation would cause both controllers to perform worse.